# Finova SmartBank System — Complete Interview Preparation Guide

## 1) Project at a Glance

• **Project type:** Full-stack digital banking demo application.
• **Backend stack:** Java 21, Spring Boot 4, Spring Security, Spring Data JPA, PostgreSQL.
• **Frontend stack:** Thymeleaf templates + vanilla JS/CSS.
• **API docs:** Springdoc OpenAPI (Swagger, configurable).
• **AI stack:** Spring AI + Google Gemini chat + Gemini embeddings + RAG using local policy documents.
• **Authentication:** JWT-based stateless authentication with role-based authorization.

---

## 2) Core Business Features

### A. Authentication & Identity

1. **User Registration (`/api/auth/register`)**
   - Validates name, email, phone, password.
   - Normalizes email (trim/lowercase).
   - Prevents duplicate email registration.
   - Hashes password using BCrypt.
   - Auto-assigns `CUSTOMER` role.
   - Auto-creates a default `SAVINGS` account after registration.

2. **User Login (`/api/auth/login`)**
   - Uses Spring AuthenticationManager for credential validation.
   - Rejects blocked users.
   - Issues JWT with email as subject and role claim.

3. **Admin Seeding at Startup**
   - Optional admin creation from environment variables.
   - Skips seeding if credentials are absent.
   - Prevents replacing a non-admin existing user.

### B. Account Management

1. **Open account (`POST /api/accounts`)**
   - Allows account creation by account type (`SAVINGS`, `CURRENT`).
   - Prevents duplicate account type per customer.

2. **View accounts (`GET /api/accounts`, `GET /api/accounts/{id}`)**
   - Customer can only access own accounts.

3. **Account states**
   - `ACTIVE`, `BLOCKED`, `CLOSED`.

### C. Banking Transactions

1. **Deposit** (`POST /api/accounts/{id}/deposit`)
2. **Withdraw** (`POST /api/accounts/{id}/withdraw`)
3. **Transfer** (`POST /api/accounts/{id}/transfer`)

Important transaction logic:
• Amount precision is controlled (scale 2).

• Insufficient balance checks are enforced.
• Account ownership + active-status checks are enforced.
• Transfer uses pessimistic DB locking and deterministic lock order to avoid deadlocks.
• Transfer creates paired entries (`TRANSFER_DEBIT`, `TRANSFER_CREDIT`) with same reference.

### D. Transaction History

1. **Search transaction history (`GET /api/transactions`)**
   - Filters: accountId, type, from, to, page, size.
   - Date range validation (`from <= to`).
   - Inclusive date behavior for `to` day.
   - Pagination and page-size capping.

2. **Transaction details (`GET /api/transactions/{id}`)**
   - Ensures customer can access only own transaction.

### E. Beneficiary Management

1. **Add beneficiary (`POST /api/beneficiaries`)**
   - Validates account number and bank name.
   - Prevents duplicate active beneficiary per account number.

2. **List beneficiaries (`GET /api/beneficiaries`)**
3. **Remove beneficiary (`DELETE /api/beneficiaries/{id}`)**
   - Uses soft delete (`INACTIVE`) instead of hard delete.

### F. Credit Card Module

1. **Request card (`POST /api/cards/request`)**
   - Card types: `STANDARD`, `GOLD`, `PLATINUM` with different default limits.
   - Creates masked card number (`**** **** **** 1234`).
   - Initial status `INACTIVE`, 5-year expiry.

2. **Card lifecycle**
   - List/get cards.
   - Activate card (only from INACTIVE).
   - Block card (from INACTIVE/ACTIVE).

3. **Card purchase (`POST /api/cards/{id}/purchase`)**
   - Requires card usability + non-expired card.
   - Deducts available credit, adds outstanding amount.

4. **Card payment (`POST /api/cards/{id}/payments`)**
   - Pays card dues from customer bank account.
   - Checks account status and balance.
   - Reduces outstanding amount, restores available credit.
   - Also records a linked bank transaction of type `CARD_PAYMENT`.

5. **Card statements**
   - Purchases history and payment history per card.

### G. Admin Analytics

Protected by `ROLE_ADMIN`:

1. **Dashboard metrics (`/api/admin/dashboard`)**
   - Total customers/accounts/cards.
   - Total account balances.

- Total card outstanding amount.
- Today transaction count.

2. **Statistics (`/api/admin/statistics`)**
   - Type-wise transaction counts and sums.
   - Last 7-day daily summary (including zero-filled missing days).
   - Account-type and card-status distributions.

3. **Recent transactions (`/api/admin/recent-transactions`)**
   - Limited and bounded pagination.
   - Account numbers are masked in response.

### H. AI Assistant + RAG

1. **Ask endpoint (`POST /api/ai/ask`)**
   - Authenticated customer asks banking questions.

2. **RAG retrieval flow**
   - Retrieves top policy chunks from vector store by similarity threshold.
   - Builds context + user question prompt.
   - Uses Gemini chat model to answer.

3. **Policy-grounded behavior**
   - Assistant is constrained by system instructions:
     - No fabricated SmartBank policies.
     - No operational account/card actions.
     - No secret request behavior.
     - Limited response size.

4. **Rate limiting for AI endpoint**
   - Per-user minute and daily limits.
   - In-memory sliding window algorithm.

5. **Knowledge ingestion**
   - Reads text policy files from `knowledge-base/**/*.txt`.
   - Chunks by headings/sections.
   - Embeds in batches.
   - Saves vector store + fingerprint hash.
   - Reuses saved vectors if file content/model/chunking unchanged.

---

## 3) Frontend Features (Thymeleaf UI)

• Public pages: login, register.
• Customer pages: dashboard, accounts, transactions, beneficiaries, cards, AI assistant, profile.
• Admin page: admin dashboard.
• Frontend acts as a shell; secured API calls are authenticated with JWT.

---

## 4) Security Features

1. **Stateless JWT security**
   - CSRF disabled for API context.

- Session creation policy set to stateless.
- JWT filter executes before username/password filter.

2. **Route authorization**
- Public: auth endpoints, Swagger, static assets, page shells.
- Admin APIs: `hasRole("ADMIN")`.
- Others require authentication.

3. **Credential security**
- BCrypt password hashing.
- JWT secret and expiry from environment configuration.

4. **Blocked-user support**
- UserDetails marks blocked users as disabled.

5. **Input validation**
- Bean validation on DTOs (email format, phone/account patterns, min amounts, required fields).

6. **Error handling**
- Global exception handling for validation, auth, business rules, resource not found, malformed body, and AI unavailability.

---

## 5) Data Model Concepts

Main entities:
• `Customer` (role/status based identity)
• `Account` (account type, balance, status)
• `Transaction` (type, amount, reference, status, timestamp)
• `Beneficiary` (soft-deletable external payee)
• `CreditCard` (credit limit model, card status, expiry)
• `CardTransaction` (merchant spends)
• `CardPayment` (repayments)

Enums encode business states and behavior:
• Role, account type/status, customer status, card type/status, beneficiary status, transaction type/status.

---

## 6) Practical API Concepts Interviewers Can Ask

### Auth & JWT
• How JWT is generated, what claims are inside, where token expiry is configured.
• Why stateless auth scales better than session-based auth for APIs.
• How blocked users are denied login.

### Transaction Integrity
• Why `@Transactional` is used.
• Why pessimistic locking is used for money movement.
• How lock ordering prevents deadlocks in opposite-direction transfers.
• Why transfer creates debit and credit records.

### Validation & Error Semantics
• Difference between validation errors (400), auth errors (401/403), business rule failures (422), not found (404), conflicts (409).
• Why strong DTO validation prevents invalid states early.

### Card Flows
• Difference between available credit and outstanding amount.
• How purchase and payment mutate card state.
• Why card status and expiry checks are mandatory.

### AI/RAG
• Difference between a plain LLM call vs retrieval-augmented generation.
• Why policy grounding reduces hallucination.
• Why embeddings are cached/persisted.
• Trade-offs of in-memory rate limiter and in-memory vector store.

### Admin Analytics
• Aggregation query patterns and grouped statistics.
• Why masking account numbers in admin responses matters.

---

## 7) End-to-End Request Flow (What to Explain in Interview)
1. Client sends HTTP request with JWT.
2. `JwtAuthenticationFilter` extracts ****** and authenticates user context.
3. Controller reads authenticated email (`Authentication#getName`).
4. Service layer enforces business and ownership checks.
5. Repository performs JPA operations (with optional row locks).
6. DTO response returned as JSON.
7. Any exception is transformed into structured `ErrorResponse` by global handler.

---

## 8) Database/Concurrency Design Points
• JPA entities include indexes on commonly queried foreign keys.
• Money stored as `BigDecimal` with precision/scale.
• Ownership checks prevent horizontal privilege escalation.
• Pessimistic locks protect consistency in concurrent debit/transfer/card payment operations.

---

## 9) Configuration & Deployment Topics
• Externalized env-based configuration (`DB`, `JWT`, `admin seed`, `Gemini key`).
• Swagger toggling via `SWAGGER_ENABLED`.
• Port override via env.
• RAG store path configurable and persisted under user home.

---

## 10) Practical Hands-on Questions Interviewers May Ask
1. "How would you prevent double spending in concurrent withdrawals?"

2. "How is deadlock avoided in transfer logic?"
3. "How do you enforce account ownership for every API?"
4. "Where can this project be vulnerable if validation is missed?"
5. "Why choose JWT over server sessions?"
6. "How would you move from single-instance in-memory limiter to distributed rate limit?"
7. "How would you test transfer atomicity?"
8. "How would you extend card module for EMI or billing cycles?"
9. "How would you support idempotency for transaction APIs?"
10. "How do you secure AI features from prompt injection and policy leakage?"

---

## 11) High-Value Improvement Ideas (Good Interview Discussion)

• Add integration/unit test coverage for all service flows and edge cases.
• Add refresh tokens and token revocation strategy.
• Add distributed cache (Redis) for rate limiting.
• Add audit trail and observability dashboards.
• Add idempotency keys for transaction APIs.
• Add stronger fraud/risk checks and velocity rules.
• Introduce event-driven architecture for ledger-grade transaction processing.

---

## 12) Quick Revision Sheet

• Stateless auth: JWT + role-based route protection.
• Strong service-layer ownership checks.
• Transactional boundaries + pessimistic locks for monetary consistency.
• Full digital banking flows: account, transfer, beneficiary, cards, payments.
• Admin analytics and masked sensitive outputs.
• AI assistant grounded through RAG + policy files + rate limits.

---

## 13) 5-Minute Interview Pitch

"Finova SmartBank is a Spring Boot based digital banking system with JWT authentication, role-based authorization, and a complete customer banking workflow: account creation, deposits, withdrawals, transfers, beneficiary management, credit card lifecycle, card purchases, and repayment flows.

A key strength is transaction safety: service methods are transactional, money operations use pessimistic locking, and transfer logic acquires locks in deterministic order to avoid deadlocks. The project also includes an admin analytics layer with grouped transaction insights and masked account data.

It additionally integrates an AI assistant using Spring AI + Gemini with retrieval-augmented generation over internal banking policy documents, enforcing grounded responses and rate limiting. So the project demonstrates both core banking backend engineering and modern AI integration patterns in one system."